Section 8.3 - Leveraging Pretrained Models

November 28, 2025

1 Feature extraction with a pretrained model

- ConvNet = Convolutional Base + Classifier
- Feature extraction: Reuse the Convolutional Base, containing more generic representations
- Adapt for different CV problems
- The new class set needs not overlap with the original class set

1.1 Small dataset

1) Dog-vs-cat classification: 1 = dog & 0 = cat
2) Balanced dataset: 2.000 Training + 1.000 Validation + 2.000 Testing
3) Data processing pipeline: image_dataset_from_directory()

- Read JPEG picture files
- Decode JPEG content to RGB grids of pixels
- Convert into floating-point tensors
- Resize to 180 x 180 x 3
- Pack in batches

[1]:
```
from pathlib import Path
from tensorflow.keras.utils import image_dataset_from_directory

new_base_dir = Path("cats_vs_dogs_small")

train_dataset = image_dataset_from_directory(
    new_base_dir / "train",
    image_size=(180, 180),
    batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)
```

```
Found 2000 files belonging to 2 classes.
Found 1000 files belonging to 2 classes.
Found 2000 files belonging to 2 classes.
```

1.2 Pretrained model & Original dataset

- Pretrained model: VGG16 packaged with Keras
- Original dataset: ImageNet - 1.4 millions images of 1,000 classes
- Original dataset: large & general enough
- Feature maps of pretrained models: generic concepts over a picture

[2]:
```
from tensorflow import keras
from tensorflow.keras import layers

conv_base = keras.applications.vgg16.VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(180, 180, 3))

# Inspect the convolutional base
conv_base.summary()
```

```
Model: "vgg16"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 180, 180, 3)]     0

 block1_conv1 (Conv2D)       (None, 180, 180, 64)      1792

 block1_conv2 (Conv2D)       (None, 180, 180, 64)      36928

 block1_pool (MaxPooling2D)  (None, 90, 90, 64)        0

 block2_conv1 (Conv2D)       (None, 90, 90, 128)       73856

 block2_conv2 (Conv2D)       (None, 90, 90, 128)       147584

 block2_pool (MaxPooling2D)  (None, 45, 45, 128)       0

 block3_conv1 (Conv2D)       (None, 45, 45, 256)       295168

 block3_conv2 (Conv2D)       (None, 45, 45, 256)       590080

 block3_conv3 (Conv2D)       (None, 45, 45, 256)       590080

 block3_pool (MaxPooling2D)  (None, 22, 22, 256)       0
```

```
 block4_conv1 (Conv2D)       (None, 22, 22, 512)       1180160

 block4_conv2 (Conv2D)       (None, 22, 22, 512)       2359808

 block4_conv3 (Conv2D)       (None, 22, 22, 512)       2359808

 block4_pool (MaxPooling2D)  (None, 11, 11, 512)       0

 block5_conv1 (Conv2D)       (None, 11, 11, 512)       2359808

 block5_conv2 (Conv2D)       (None, 11, 11, 512)       2359808

 block5_conv3 (Conv2D)       (None, 11, 11, 512)       2359808

 block5_pool (MaxPooling2D)  (None, 5, 5, 512)         0

=================================================================
Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0
_________________________________________________________________
```

1.3 Extract VGG16 features & labels

- Early layers: highly generic features (edges, color, textures, etc)
- Higher layers: more abstract concepts (cat ear, dog eye, etc)
- Notice the preprocess_input() layer for vgg16

[3]:
```
import numpy as np

def get_features_and_labels(dataset):
    all_features = []
    all_labels = []
    for images, labels in dataset:
        preprocessed_images = keras.applications.vgg16.preprocess_input(images)
        features = conv_base.predict(preprocessed_images)
        all_features.append(features)
        all_labels.append(labels)
    return np.concatenate(all_features), np.concatenate(all_labels)

train_features, train_labels =  get_features_and_labels(train_dataset)
val_features, val_labels =  get_features_and_labels(validation_dataset)
test_features, test_labels =  get_features_and_labels(test_dataset)
```

```
1/1 [==============================] - 2s 2s/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 20ms/step
```

```
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 22ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 22ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 22ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 23ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
```

```
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 1s 1s/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 1s 593ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 24ms/step
1/1 [==============================] - 0s 18ms/step
```

```
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 21ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 22ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 23ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 20ms/step
```

```
1/1 [==============================] - 0s 20ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 18ms/step
1/1 [==============================] - 0s 19ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 17ms/step
1/1 [==============================] - 0s 16ms/step
1/1 [==============================] - 0s 16ms/step
```

[4]:
```
# check the dataset shapes
print(train_features.shape, train_labels.shape)
print(val_features.shape, val_labels.shape)
print(test_features.shape, test_labels.shape)
```

```
(2000, 5, 5, 512) (2000,)
(1000, 5, 5, 512) (1000,)
(2000, 5, 5, 512) (2000,)
```

1.4 The classifier

- The new Classifier works on the transformed datasets
- Does not allow image augmentation!
- Alternative: New Model = Convolutional Base + new Classifier

[5]:
```
inputs = keras.Input(shape=(5, 5, 512))
x = layers.Flatten()(inputs)
x = layers.Dense(256)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)

# Inspect the model
model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 5, 5, 512)]       0

 flatten (Flatten)           (None, 12800)             0

 dense (Dense)               (None, 256)               3277056

 dropout (Dropout)           (None, 256)               0

 dense_1 (Dense)             (None, 1)                 257
```

```
=================================================================
Total params: 3,277,313
Trainable params: 3,277,313
Non-trainable params: 0
_________________________________________________________________
```

[6]:
```
# Training
model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
      filepath="feature_extraction.h5",
      save_best_only=True,
      monitor="val_loss")
]

history = model.fit(
    train_features, train_labels,
    epochs=20,
    validation_data=(val_features, val_labels),
    callbacks=callbacks)
```

```
Epoch 1/20
63/63 [==============================] - 1s 7ms/step - loss: 19.9868 - accuracy:
0.9205 - val_loss: 2.3160 - val_accuracy: 0.9760
Epoch 2/20
63/63 [==============================] - 0s 4ms/step - loss: 4.1928 - accuracy:
0.9705 - val_loss: 2.2931 - val_accuracy: 0.9790
Epoch 3/20
63/63 [==============================] - 0s 4ms/step - loss: 1.9373 - accuracy:
0.9845 - val_loss: 7.1430 - val_accuracy: 0.9650
Epoch 4/20
63/63 [==============================] - 0s 4ms/step - loss: 1.3733 - accuracy:
0.9895 - val_loss: 4.9930 - val_accuracy: 0.9710
Epoch 5/20
63/63 [==============================] - 0s 4ms/step - loss: 0.6600 - accuracy:
0.9940 - val_loss: 4.4326 - val_accuracy: 0.9690
Epoch 6/20
63/63 [==============================] - 0s 4ms/step - loss: 0.8261 - accuracy:
0.9905 - val_loss: 5.5059 - val_accuracy: 0.9730
Epoch 7/20
63/63 [==============================] - 0s 4ms/step - loss: 0.6232 - accuracy:
0.9945 - val_loss: 5.8711 - val_accuracy: 0.9710
Epoch 8/20
63/63 [==============================] - 0s 4ms/step - loss: 0.6814 - accuracy:
```

```
0.9965 - val_loss: 5.3267 - val_accuracy: 0.9730
Epoch 9/20
63/63 [==============================] - 0s 4ms/step - loss: 0.1942 - accuracy:
0.9960 - val_loss: 4.7248 - val_accuracy: 0.9800
Epoch 10/20
63/63 [==============================] - 0s 4ms/step - loss: 0.2879 - accuracy:
0.9960 - val_loss: 3.7082 - val_accuracy: 0.9790
Epoch 11/20
63/63 [==============================] - 0s 3ms/step - loss: 0.2629 - accuracy:
0.9970 - val_loss: 4.0531 - val_accuracy: 0.9810
Epoch 12/20
63/63 [==============================] - 0s 4ms/step - loss: 0.1058 - accuracy:
0.9990 - val_loss: 4.5334 - val_accuracy: 0.9790
Epoch 13/20
63/63 [==============================] - 0s 3ms/step - loss: 0.1347 - accuracy:
0.9980 - val_loss: 4.5895 - val_accuracy: 0.9780
Epoch 14/20
63/63 [==============================] - 0s 4ms/step - loss: 4.0153e-31 -
accuracy: 1.0000 - val_loss: 4.5895 - val_accuracy: 0.9780
Epoch 15/20
63/63 [==============================] - 0s 4ms/step - loss: 0.0882 - accuracy:
0.9990 - val_loss: 4.8473 - val_accuracy: 0.9770
Epoch 16/20
63/63 [==============================] - 0s 4ms/step - loss: 0.0781 - accuracy:
0.9990 - val_loss: 4.8333 - val_accuracy: 0.9740
Epoch 17/20
63/63 [==============================] - 0s 4ms/step - loss: 4.8845e-33 -
accuracy: 1.0000 - val_loss: 4.8333 - val_accuracy: 0.9740
Epoch 18/20
63/63 [==============================] - 0s 4ms/step - loss: 0.1190 - accuracy:
0.9980 - val_loss: 5.7643 - val_accuracy: 0.9730
Epoch 19/20
63/63 [==============================] - 0s 4ms/step - loss: 0.0886 - accuracy:
0.9990 - val_loss: 4.9766 - val_accuracy: 0.9800
Epoch 20/20
63/63 [==============================] - 0s 4ms/step - loss: 0.1821 - accuracy:
0.9990 - val_loss: 4.5032 - val_accuracy: 0.9810
```

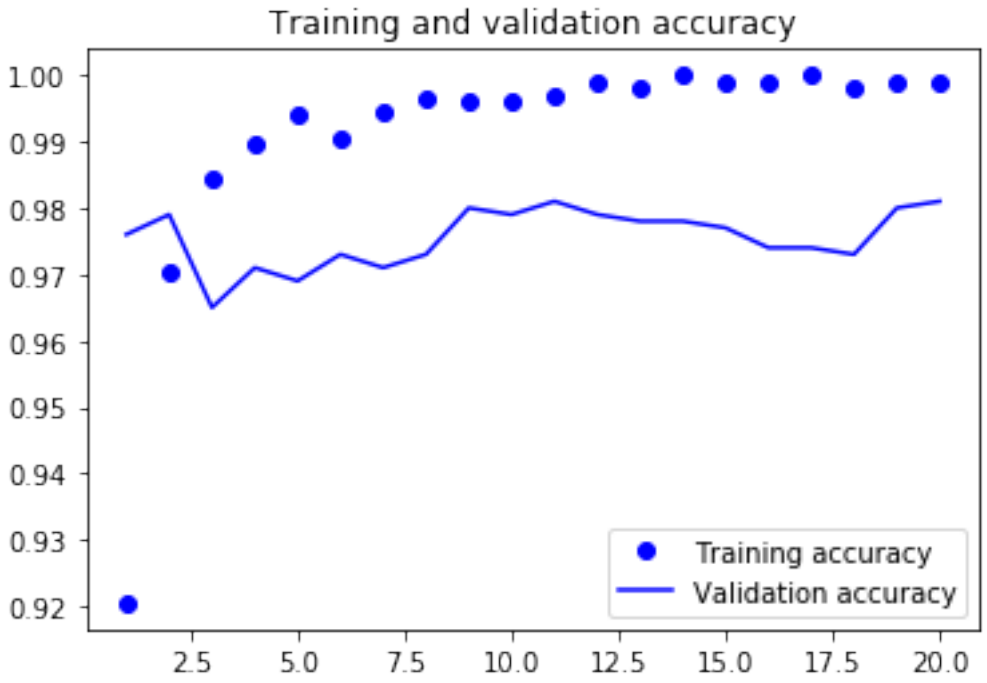
1.5 Plotting the training results

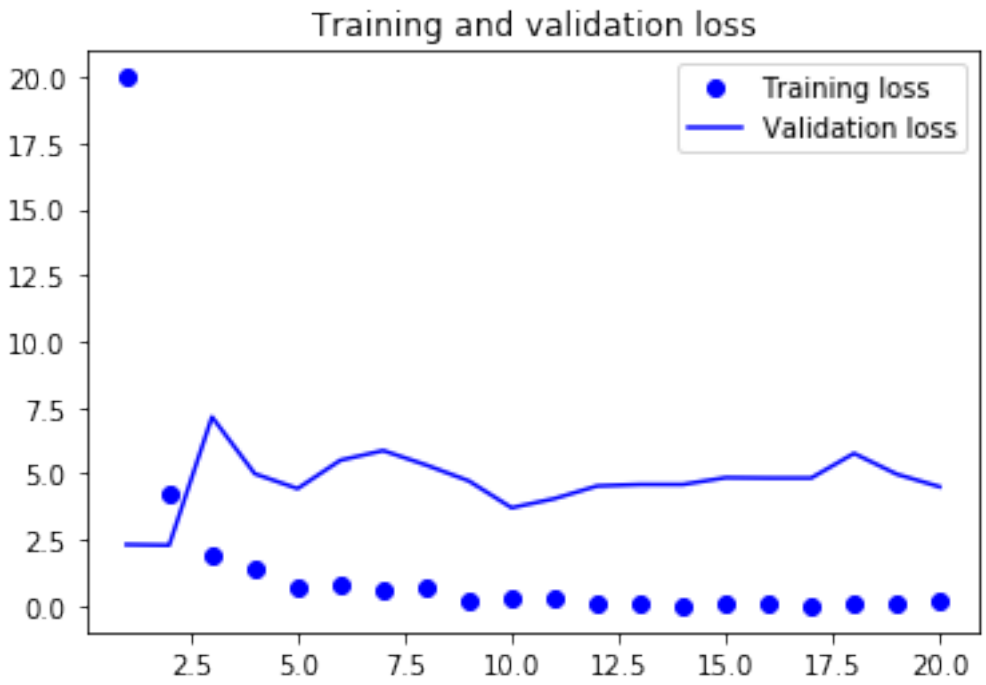
- Overfitting appears after 4 epochs

[7]:
```python
import matplotlib.pyplot as plt

acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
```

```
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





1.6 Evaluation on the test dataset

- Accuracy: 97% (Previous accuracy: 78%)

[8]:
```
test_model = keras.models.load_model(
    "feature_extraction.h5")
test_loss, test_acc = test_model.evaluate(test_features, test_labels)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [==============================] - 0s 2ms/step - loss: 4.1360 - accuracy:
0.9730
Test accuracy: 0.973
```

2 Feature extraction & data augmentation

2.1 Freeze the VGG16 convolutional base

- Freeze the conv_base
- len(model.trainable_weights) –> number of weight tensors
- Each Conv2D has two trainable variables/tensors (not scalar weights)

[9]:
```
conv_base  = keras.applications.vgg16.VGG16(
    weights="imagenet",
    include_top=False)
conv_base.trainable = False
```

```
# number of weight tensors
print("Number of trainable weights:", len(conv_base.trainable_weights))
```

```
Number of trainable weights: 0
```

2.2 Model with image augmentation

1) Purpose: Mitigate overfitting by adding useful representations
2) Random transformations: Generate believable looking images

- RandomFlip "horizontal"
- RandomRotation [-36, 36] degrees
- RandomZoom [-20%, 20%]

[10]:
```
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.2),
    ]
)

# model with image augmentation
inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)
x = keras.applications.vgg16.preprocess_input(x)
x = conv_base(x)
x = layers.Flatten()(x)
x = layers.Dense(256)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
image_augmented_model = keras.Model(inputs, outputs, name =␣
 ↪"feature_extraction_image_augmented_model")

# Inspect the new model
image_augmented_model.summary()
```

```
Model: "feature_extraction_image_augmented_model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_4 (InputLayer)        [(None, 180, 180, 3)]     0

 sequential (Sequential)     (None, 180, 180, 3)       0

 tf.__operators__.getitem (S  (None, 180, 180, 3)      0
 licingOpLambda)
```

```
 tf.nn.bias_add (TFOpLambda)  (None, 180, 180, 3)      0

 vgg16 (Functional)           (None, None, None, 512)   14714688

 flatten_1 (Flatten)          (None, 12800)             0

 dense_2 (Dense)              (None, 256)               3277056

 dropout_1 (Dropout)          (None, 256)               0

 dense_3 (Dense)              (None, 1)                 257

=================================================================
Total params: 17,992,001
Trainable params: 3,277,313
Non-trainable params: 14,714,688
_________________________________________________________________
```

[11]:
```
# number of weight tensors
print("Number of trainable weights:", len(image_augmented_model.
  ↪trainable_weights))
```

```
Number of trainable weights: 4
```

2.3 Training the new model

[12]:
```
# Hide the warning messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

image_augmented_model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="feature_extraction_with_data_augmentation.h5",
        save_best_only=True,
        monitor="val_loss")
]

history = image_augmented_model.fit(
    train_dataset,
    epochs=50,
    validation_data=validation_dataset,
```

```
    callbacks=callbacks)
```

```
Epoch 1/50
63/63 [==============================] - 11s 142ms/step - loss: 16.8588 -
accuracy: 0.8975 - val_loss: 9.4224 - val_accuracy: 0.9470
Epoch 2/50
63/63 [==============================] - 9s 140ms/step - loss: 6.7458 -
accuracy: 0.9490 - val_loss: 5.8407 - val_accuracy: 0.9590
Epoch 3/50
63/63 [==============================] - 9s 141ms/step - loss: 6.0255 -
accuracy: 0.9530 - val_loss: 5.5990 - val_accuracy: 0.9610
Epoch 4/50
63/63 [==============================] - 9s 139ms/step - loss: 4.3775 -
accuracy: 0.9690 - val_loss: 7.0404 - val_accuracy: 0.9600
Epoch 5/50
63/63 [==============================] - 9s 142ms/step - loss: 3.3478 -
accuracy: 0.9670 - val_loss: 3.4566 - val_accuracy: 0.9720
Epoch 6/50
63/63 [==============================] - 9s 140ms/step - loss: 2.6198 -
accuracy: 0.9745 - val_loss: 7.0179 - val_accuracy: 0.9680
Epoch 7/50
63/63 [==============================] - 9s 140ms/step - loss: 3.4977 -
accuracy: 0.9715 - val_loss: 4.9128 - val_accuracy: 0.9710
Epoch 8/50
63/63 [==============================] - 9s 140ms/step - loss: 2.4812 -
accuracy: 0.9805 - val_loss: 3.8273 - val_accuracy: 0.9740
Epoch 9/50
63/63 [==============================] - 9s 141ms/step - loss: 2.9536 -
accuracy: 0.9760 - val_loss: 4.4756 - val_accuracy: 0.9730
Epoch 10/50
63/63 [==============================] - 9s 142ms/step - loss: 1.3159 -
accuracy: 0.9850 - val_loss: 3.3413 - val_accuracy: 0.9790
Epoch 11/50
63/63 [==============================] - 9s 140ms/step - loss: 2.3605 -
accuracy: 0.9790 - val_loss: 3.9457 - val_accuracy: 0.9780
Epoch 12/50
63/63 [==============================] - 9s 143ms/step - loss: 1.1696 -
accuracy: 0.9855 - val_loss: 3.3382 - val_accuracy: 0.9790
Epoch 13/50
63/63 [==============================] - 9s 142ms/step - loss: 1.6909 -
accuracy: 0.9825 - val_loss: 6.6209 - val_accuracy: 0.9650
Epoch 14/50
63/63 [==============================] - 9s 141ms/step - loss: 0.9802 -
accuracy: 0.9875 - val_loss: 3.6743 - val_accuracy: 0.9810
Epoch 15/50
63/63 [==============================] - 9s 142ms/step - loss: 1.5179 -
accuracy: 0.9880 - val_loss: 3.6806 - val_accuracy: 0.9790
Epoch 16/50
```

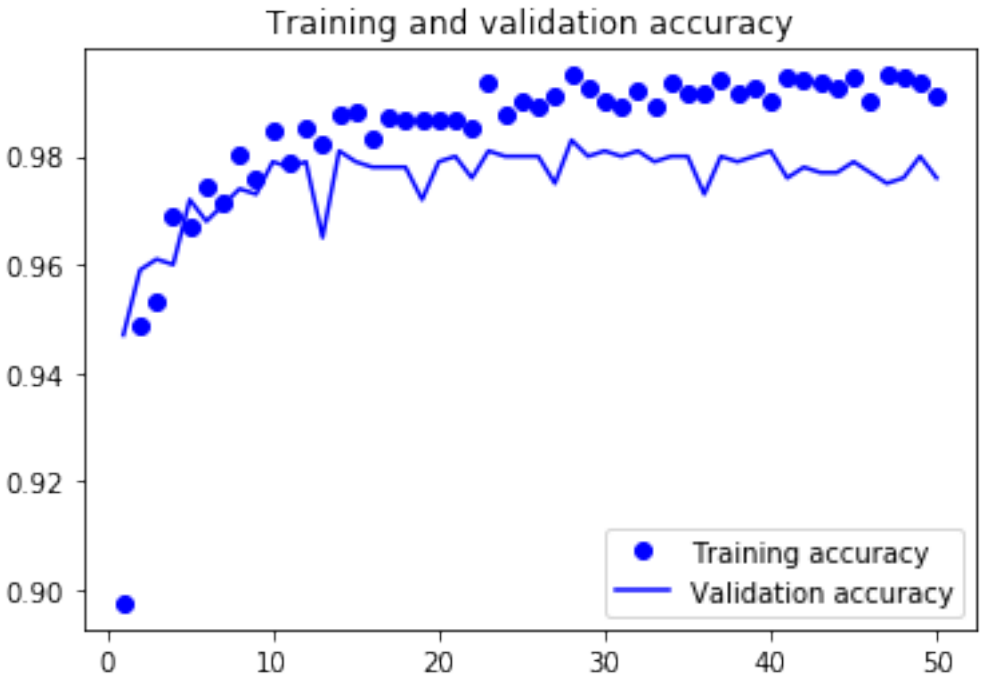
```
63/63 [==============================] - 9s 141ms/step - loss: 1.4690 -
accuracy: 0.9835 - val_loss: 4.2324 - val_accuracy: 0.9780
Epoch 17/50
63/63 [==============================] - 9s 142ms/step - loss: 0.9746 -
accuracy: 0.9870 - val_loss: 3.7988 - val_accuracy: 0.9780
Epoch 18/50
63/63 [==============================] - 9s 141ms/step - loss: 1.6329 -
accuracy: 0.9865 - val_loss: 3.7305 - val_accuracy: 0.9780
Epoch 19/50
63/63 [==============================] - 9s 141ms/step - loss: 1.0189 -
accuracy: 0.9865 - val_loss: 4.6428 - val_accuracy: 0.9720
Epoch 20/50
63/63 [==============================] - 9s 142ms/step - loss: 1.1672 -
accuracy: 0.9865 - val_loss: 4.1065 - val_accuracy: 0.9790
Epoch 21/50
63/63 [==============================] - 9s 144ms/step - loss: 1.2144 -
accuracy: 0.9865 - val_loss: 2.9670 - val_accuracy: 0.9800
Epoch 22/50
63/63 [==============================] - 9s 144ms/step - loss: 1.4471 -
accuracy: 0.9855 - val_loss: 2.8544 - val_accuracy: 0.9760
Epoch 23/50
63/63 [==============================] - 9s 145ms/step - loss: 0.6382 -
accuracy: 0.9935 - val_loss: 2.3508 - val_accuracy: 0.9810
Epoch 24/50
63/63 [==============================] - 9s 142ms/step - loss: 0.7025 -
accuracy: 0.9875 - val_loss: 2.8451 - val_accuracy: 0.9800
Epoch 25/50
63/63 [==============================] - 9s 142ms/step - loss: 0.6288 -
accuracy: 0.9900 - val_loss: 2.3641 - val_accuracy: 0.9800
Epoch 26/50
63/63 [==============================] - 9s 141ms/step - loss: 0.7355 -
accuracy: 0.9890 - val_loss: 3.8935 - val_accuracy: 0.9800
Epoch 27/50
63/63 [==============================] - 9s 142ms/step - loss: 0.8011 -
accuracy: 0.9910 - val_loss: 3.8990 - val_accuracy: 0.9750
Epoch 28/50
63/63 [==============================] - 9s 144ms/step - loss: 0.3177 -
accuracy: 0.9950 - val_loss: 2.3124 - val_accuracy: 0.9830
Epoch 29/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5802 -
accuracy: 0.9925 - val_loss: 3.0879 - val_accuracy: 0.9800
Epoch 30/50
63/63 [==============================] - 9s 142ms/step - loss: 0.6259 -
accuracy: 0.9900 - val_loss: 2.7777 - val_accuracy: 0.9810
Epoch 31/50
63/63 [==============================] - 9s 142ms/step - loss: 0.9850 -
accuracy: 0.9890 - val_loss: 2.8453 - val_accuracy: 0.9800
Epoch 32/50
```

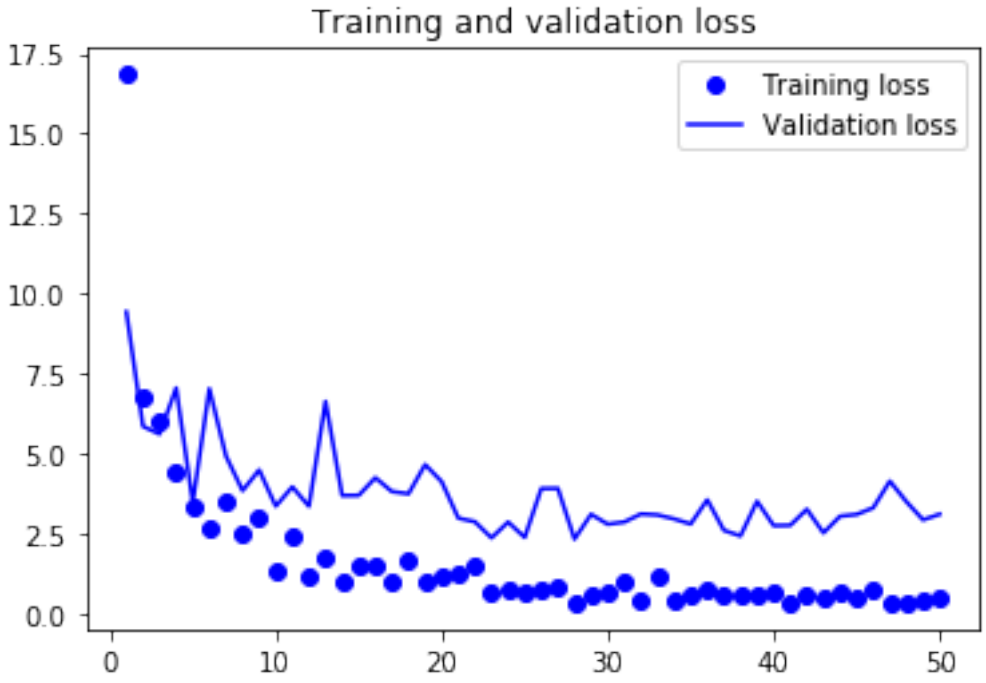
```
63/63 [==============================] - 9s 142ms/step - loss: 0.4179 -
accuracy: 0.9920 - val_loss: 3.0979 - val_accuracy: 0.9810
Epoch 33/50
63/63 [==============================] - 9s 142ms/step - loss: 1.1214 -
accuracy: 0.9890 - val_loss: 3.0689 - val_accuracy: 0.9790
Epoch 34/50
63/63 [==============================] - 9s 142ms/step - loss: 0.3719 -
accuracy: 0.9935 - val_loss: 2.9436 - val_accuracy: 0.9800
Epoch 35/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5921 -
accuracy: 0.9915 - val_loss: 2.7836 - val_accuracy: 0.9800
Epoch 36/50
63/63 [==============================] - 9s 143ms/step - loss: 0.7601 -
accuracy: 0.9915 - val_loss: 3.5421 - val_accuracy: 0.9730
Epoch 37/50
63/63 [==============================] - 9s 143ms/step - loss: 0.5205 -
accuracy: 0.9940 - val_loss: 2.5869 - val_accuracy: 0.9800
Epoch 38/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5883 -
accuracy: 0.9915 - val_loss: 2.4103 - val_accuracy: 0.9790
Epoch 39/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5209 -
accuracy: 0.9925 - val_loss: 3.4975 - val_accuracy: 0.9800
Epoch 40/50
63/63 [==============================] - 9s 142ms/step - loss: 0.6736 -
accuracy: 0.9900 - val_loss: 2.7332 - val_accuracy: 0.9810
Epoch 41/50
63/63 [==============================] - 9s 142ms/step - loss: 0.3508 -
accuracy: 0.9945 - val_loss: 2.7427 - val_accuracy: 0.9760
Epoch 42/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5362 -
accuracy: 0.9940 - val_loss: 3.2457 - val_accuracy: 0.9780
Epoch 43/50
63/63 [==============================] - 9s 143ms/step - loss: 0.5164 -
accuracy: 0.9935 - val_loss: 2.5068 - val_accuracy: 0.9770
Epoch 44/50
63/63 [==============================] - 9s 143ms/step - loss: 0.6538 -
accuracy: 0.9925 - val_loss: 3.0279 - val_accuracy: 0.9770
Epoch 45/50
63/63 [==============================] - 9s 142ms/step - loss: 0.4847 -
accuracy: 0.9945 - val_loss: 3.0827 - val_accuracy: 0.9790
Epoch 46/50
63/63 [==============================] - 9s 142ms/step - loss: 0.6868 -
accuracy: 0.9900 - val_loss: 3.2921 - val_accuracy: 0.9770
Epoch 47/50
63/63 [==============================] - 9s 142ms/step - loss: 0.3194 -
accuracy: 0.9950 - val_loss: 4.1257 - val_accuracy: 0.9750
Epoch 48/50
```

```
63/63 [==============================] - 9s 142ms/step - loss: 0.3166 -
accuracy: 0.9945 - val_loss: 3.4817 - val_accuracy: 0.9760
Epoch 49/50
63/63 [==============================] - 9s 142ms/step - loss: 0.3991 -
accuracy: 0.9935 - val_loss: 2.9139 - val_accuracy: 0.9800
Epoch 50/50
63/63 [==============================] - 9s 142ms/step - loss: 0.5047 -
accuracy: 0.9910 - val_loss: 3.0896 - val_accuracy: 0.9760
```

2.4 Plot the training results

[13]:
```
accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





2.5 Evaluation on the test dataset

- Accuracy: 97.5%

[14]:
```
test_model = keras.models.load_model(
    "feature_extraction_with_data_augmentation.h5")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [==============================] - 4s 65ms/step - loss: 4.0420 - accuracy:
0.9750
Test accuracy: 0.975
```